

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Реализация БД в рамках СУБД»

Выполнили:

Шибает Иван Дмитриевич, студент группы N3352

(подпись)

Малышев Григорий Игоревич, студент группы
N3352

(подпись)

Проверил:

Ярцева Наталия Андреевна

(подпись)

СОДЕРЖАНИЕ

Введение	3
1 Выбор СУБД	4
2 Создание базы данных и таблиц	5
3 Установка взаимосвязей между таблицами	8
4 Индексирование таблиц	10
5 Модификация структуры таблицы	13
6 Заполнение базы данных	15
7 Создание представлений	21
8 Проверка ограничений целостности	25
Заключение	26
Список использованных источников	27

ВВЕДЕНИЕ

Цель работы — получение навыков работы с современными системами управления базами данных [1].

Задание: выбрать СУБД и обосновать выбор; создать в ней базу данных по итоговой схеме отношений из лабораторной работы № 1; заполнить основные таблицы не менее чем семью-восемью кортежами; привести код создания таблиц, внесения данных и модификации структуры; проиндексировать атрибуты соединений и поиска; установить связи между таблицами; создать представления, спроектированные в пункте 5 лабораторной работы № 1.

Предметная область сохранена из первой работы — информационная система мастерской реставрации музыкальных инструментов. Схема включает девять отношений: восемь самостоятельных сущностей и ассоциативное отношение «order_work». Все имена и контакты в демонстрационных данных вымышлены.

1 Выбор СУБД

Для реализации выбрана СУБД PostgreSQL 14 (сборка Homebrew 14.21 для macOS). Выбор определяется требованиями не только этой, но и следующих работ курса:

- серверная модель ролей и привилегий «GRANT»/«REVOKE», в том числе на уровне отдельных столбцов, необходима для разграничения доступа в лабораторной работе № 3;
- триггеры на PL/pgSQL и расширение pgcrypto позволяют реализовать журналирование изменений и симметричное шифрование средствами самой СУБД;
- журнал предзаписи (WAL), «pg_basebackup» и восстановление на момент времени дают штатный механизм резервирования для лабораторной работы № 5;
- СУБД распространяется свободно и поддерживает ограничения целостности, частичные индексы и представления [2].

Прототип первой работы был выполнен на SQLite. SQLite не имеет серверных ролей и авторизации, поэтому для задач защиты на уровне СУБД не подходит.

Администрирование выполняется консольным клиентом psql. Для воспроизводимости используется отдельный учебный кластер: сценарий «tools/pg.sh» инициализирует его в каталоге «subjects/ibbd/.pgdata», запускает на порту 55432 и принимает подключения только с адреса 127.0.0.1. Системный сервер PostgreSQL при этом не затрагивается. Суперпользователь «postgres» входит без пароля только с loopback-адреса; прикладные роли следующих работ входят по паролю с проверкой SCRAM-SHA-256.

2 Создание базы данных и таблиц

База данных создаётся командой, приведённой в листинге 1. Сценарий «check.sh» перед созданием удаляет прежнюю копию, чтобы результат не зависел от предыдущих запусков.

```
DROP DATABASE IF EXISTS "workshop" WITH (FORCE);  
CREATE DATABASE "workshop" ENCODING 'UTF8' TEMPLATE template0;
```

Листинг 1 — Создание базы данных

Код создания таблиц приведён в листинге 2. По сравнению с прототипом на SQLite изменены типы: даты хранятся в типе «date», идентификаторы генерируются столбцами «GENERATED BY DEFAULT AS IDENTITY». Суммы по-прежнему хранятся целым числом копеек. Проверочные ограничения на допустимые статусы, неотрицательные цены и порядок дат перенесены без изменения смысла.

```
-- Отношения итоговой схемы ЛР1.  
-- Внешние ключи добавляются в 02_relations.sql.  
BEGIN;  
  
CREATE TABLE customer (  
    customer_id integer  
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    full_name text NOT NULL  
        CHECK (length(btrim(full_name)) > 0),  
    phone text NOT NULL,  
    email text  
);  
  
CREATE TABLE instrument_type (  
    type_id integer  
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    name text NOT NULL UNIQUE,  
    description text NOT NULL  
);  
  
CREATE TABLE instrument (  
    instrument_id integer  
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    customer_id integer NOT NULL,  
    type_id integer NOT NULL,
```

```

inventory_no  text NOT NULL UNIQUE,
name          text NOT NULL,
maker        text,
serial_no     text
);

CREATE TABLE employee (
    employee_id integer
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    full_name    text NOT NULL,
    specialization text NOT NULL,
    work_email   text NOT NULL UNIQUE
);

CREATE TABLE restoration_order (
    order_id      integer
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    instrument_id integer NOT NULL,
    accepted_on   date NOT NULL,
    due_on        date NOT NULL,
    status        text NOT NULL
        CHECK (status IN ('accepted', 'in_progress', 'ready',
                          'closed', 'cancelled')),
    complaint     text NOT NULL,
    CHECK (due_on >= accepted_on)
);

CREATE TABLE service (
    service_id    integer
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name          text NOT NULL UNIQUE,
    description    text NOT NULL,
    base_price_kopecks integer NOT NULL
        CHECK (base_price_kopecks >= 0)
);

CREATE TABLE order_work (
    order_id      integer NOT NULL,
    line_no        integer NOT NULL CHECK (line_no > 0),
    service_id     integer NOT NULL,
    employee_id    integer NOT NULL,
    agreed_price_kopecks integer NOT NULL
        CHECK (agreed_price_kopecks >= 0),

```

```

        status          text NOT NULL
        CHECK (status IN ('planned', 'in_progress',
                          'done', 'cancelled')),
        PRIMARY KEY (order_id, line_no)
    );

CREATE TABLE inspection (
    inspection_id integer
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    order_id      integer NOT NULL,
    employee_id   integer NOT NULL,
    inspected_on  date NOT NULL,
    conclusion     text NOT NULL
        CHECK (length(btrim(conclusion)) > 0)
);

CREATE TABLE payment (
    payment_id      integer
        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    order_id        integer NOT NULL,
    paid_on         date NOT NULL,
    amount_kopecks  integer NOT NULL
        CHECK (amount_kopecks > 0),
    method          text NOT NULL
        CHECK (method IN ('cash', 'card', 'transfer')),
    receipt_no      text NOT NULL UNIQUE
);

COMMIT;

```

Листинг 2 — Создание таблиц («sql/01_tables.sql»)

3 Установка взаимосвязей между таблицами

Внешние ключи вынесены в отдельный сценарий и добавляются командами «ALTER TABLE ... ADD CONSTRAINT» (листинг 3). Каждая связь 1:M из ER-модели первой работы реализована внешним ключом на стороне «многие» со свойством «NOT NULL». Правило «ON DELETE RESTRICT» запрещает удалять заказчика, инструмент, заказ, сотрудника или справочную запись, на которую есть ссылки.

```
-- Связи 1:M из ЛР1: внешний ключ на стороне «многие».  
-- Удаление используемой родительской записи запрещено.  
BEGIN;
```

```
ALTER TABLE instrument  
  ADD CONSTRAINT instrument_customer_fk  
    FOREIGN KEY (customer_id) REFERENCES customer  
    ON DELETE RESTRICT,  
  ADD CONSTRAINT instrument_type_fk  
    FOREIGN KEY (type_id) REFERENCES instrument_type  
    ON DELETE RESTRICT;  
  
ALTER TABLE restoration_order  
  ADD CONSTRAINT order_instrument_fk  
    FOREIGN KEY (instrument_id) REFERENCES instrument  
    ON DELETE RESTRICT;  
  
ALTER TABLE order_work  
  ADD CONSTRAINT work_order_fk  
    FOREIGN KEY (order_id) REFERENCES restoration_order  
    ON DELETE RESTRICT,  
  ADD CONSTRAINT work_service_fk  
    FOREIGN KEY (service_id) REFERENCES service  
    ON DELETE RESTRICT,  
  ADD CONSTRAINT work_employee_fk  
    FOREIGN KEY (employee_id) REFERENCES employee  
    ON DELETE RESTRICT;  
  
ALTER TABLE inspection  
  ADD CONSTRAINT inspection_order_fk  
    FOREIGN KEY (order_id) REFERENCES restoration_order  
    ON DELETE RESTRICT,  
  ADD CONSTRAINT inspection_employee_fk
```

```
FOREIGN KEY (employee_id) REFERENCES employee
ON DELETE RESTRICT;
```

```
ALTER TABLE payment
ADD CONSTRAINT payment_order_fk
FOREIGN KEY (order_id) REFERENCES restoration_order
ON DELETE RESTRICT;
```

```
COMMIT;
```

Листинг 3 — Внешние ключи («sql/02_relations.sql»)

Созданные связи прочитаны из системного каталога «pg_constraint» (листинг 4). Все девять связей модели присутствуют и имеют правило удаления RESTRICT.

conname	parent	on_delete
inspection_employee_fk	employee	RESTRICT
inspection_order_fk	restoration_order	RESTRICT
instrument_customer_fk	customer	RESTRICT
instrument_type_fk	instrument_type	RESTRICT
work_employee_fk	employee	RESTRICT
work_order_fk	restoration_order	RESTRICT
work_service_fk	service	RESTRICT
payment_order_fk	restoration_order	RESTRICT
order_instrument_fk	instrument	RESTRICT

(9 rows)

Листинг 4 — Внешние ключи в каталоге СУБД

4 Индексирование таблиц

PostgreSQL автоматически индексирует первичные ключи и ограничения UNIQUE, но не создаёт индексы для внешних ключей [3]. Поэтому индексы добавлены явно (листинг 5):

- для всех атрибутов внешних ключей, по которым соединяются таблицы в представлениях;
- для атрибутов поиска и фильтрации: ФИО и телефона заказчика, статуса и срока заказа, статуса работы, дат осмотра и оплаты.

Составной индекс «(status, due_on)» соответствует типовому запросу приёмщика: отбор активных заказов с сортировкой по сроку. Частичный уникальный индекс «order_one_active_uq» реализует ограничение первой работы: у инструмента не более одного заказа в состояниях «accepted», «in_progress», «ready».

```
-- PostgreSQL не индексирует внешние ключи автоматически.
BEGIN;

-- Атрибуты соединений (внешние ключи).
CREATE INDEX instrument_customer_idx
    ON instrument (customer_id);
CREATE INDEX instrument_type_idx ON instrument (type_id);
CREATE INDEX order_instrument_idx
    ON restoration_order (instrument_id);
CREATE INDEX work_service_idx ON order_work (service_id);
CREATE INDEX work_employee_idx ON order_work (employee_id);
CREATE INDEX inspection_order_idx ON inspection (order_id);
CREATE INDEX inspection_employee_idx
    ON inspection (employee_id);
CREATE INDEX payment_order_idx ON payment (order_id);

-- Атрибуты поиска и фильтрации.
CREATE INDEX customer_full_name_idx ON customer (full_name);
CREATE INDEX customer_phone_idx ON customer (phone);
CREATE INDEX order_status_due_idx
    ON restoration_order (status, due_on);
CREATE INDEX work_status_idx ON order_work (status);
CREATE INDEX inspection_date_idx ON inspection (inspected_on);
CREATE INDEX payment_paid_on_idx ON payment (paid_on);
```

```
-- Правило ЛР1: не более одного активного заказа инструмента.
CREATE UNIQUE INDEX order_one_active_uq
ON restoration_order (instrument_id)
WHERE status IN ('accepted', 'in_progress', 'ready');

COMMIT;
```

Листинг 5 — Индексы («sql/03_indexes.sql»)

Полный перечень индексов из представления «pg_indexes» с индексируемыми столбцами приведён в листинге 6. Индексы с суффиксами «_pkey» и «_key» созданы СУБД для первичных ключей и ограничений UNIQUE; условие частичного индекса в перечне не показано.

indexname	columns
customer_full_name_idx	(full_name)
customer_phone_idx	(phone)
customer_pkey	(customer_id)
employee_pkey	(employee_id)
employee_work_email_key	(work_email)
inspection_date_idx	(inspected_on)
inspection_employee_idx	(employee_id)
inspection_order_idx	(order_id)
inspection_pkey	(inspection_id)
instrument_customer_idx	(customer_id)
instrument_inventory_no_key	(inventory_no)
instrument_pkey	(instrument_id)
instrument_type_idx	(type_id)
instrument_type_name_key	(name)
instrument_type_pkey	(type_id)
order_work_pkey	(order_id, line_no)
work_employee_idx	(employee_id)
work_service_idx	(service_id)
work_status_idx	(status)
payment_order_idx	(order_id)
payment_paid_on_idx	(paid_on)
payment_pkey	(payment_id)
payment_receipt_no_key	(receipt_no)
order_instrument_idx	(instrument_id)
order_one_active_uq	(instrument_id)
order_status_due_idx	(status, due_on)
restoration_order_pkey	(order_id)

```

service_name_key          | (name)
service_pkey              | (service_id)
(29 rows)

```

Листинг 6 — Индексы базы данных

На восьми строках планировщик предпочитает последовательное чтение, поэтому для проверки применимости индексов оно запрещено параметром «enable_seqscan = off». Планы в листинге 7 показывают, что отбор заказов по статусу использует индекс «order_status_due_idx», выбор оплат заказа по внешнему ключу — «payment_order_idx», а поиск заказчика по телефону — «customer_phone_idx».

QUERY PLAN

```

-----
Sort
  Sort Key: due_on
  -> Bitmap Heap Scan on restoration_order
        Recheck Cond: (status = 'in_progress'::text)
        -> Bitmap Index Scan on order_status_due_idx
              Index Cond: (status = 'in_progress'::text)
(6 rows)

```

QUERY PLAN

```

-----
Bitmap Heap Scan on payment
  Recheck Cond: (order_id = 1)
  -> Bitmap Index Scan on payment_order_idx
        Index Cond: (order_id = 1)
(4 rows)

```

QUERY PLAN

```

-----
Bitmap Heap Scan on customer
  Recheck Cond: (phone = '+7 900 000-00-04'::text)
  -> Bitmap Index Scan on customer_phone_idx
        Index Cond: (phone = '+7 900 000-00-04'::text)
(4 rows)

```

Листинг 7 — Планы выполнения запросов

5 Модификация структуры таблицы

Команда модификации структуры добавляет в таблицу заказов дату фактической выдачи инструмента «closed_on» и ограничение, связывающее её со статусом (листинг 8). Дата обязательна для закрытого заказа, не может быть раньше даты приёма и должна отсутствовать у незакрытых заказов [4].

```
-- Модификация структуры: дата фактической выдачи инструмента.  
-- Только для закрытого заказа и не раньше даты приёма.
```

```
BEGIN;
```

```
ALTER TABLE restoration_order  
  ADD COLUMN closed_on date,  
  ADD CONSTRAINT order_closed_on_check CHECK (  
    (status = 'closed' AND closed_on IS NOT NULL  
      AND closed_on >= accepted_on)  
    OR (status <> 'closed' AND closed_on IS NULL)  
  );
```

```
COMMENT ON COLUMN restoration_order.closed_on  
  IS 'Дата выдачи инструмента заказчику';
```

```
COMMIT;
```

Листинг 8 — Модификация структуры («sql/04_alter.sql»)

Результат модификации — новый необязательный столбец в каталоге и заполненные даты у двух закрытых заказов — показан в листинге 9.

column_name	data_type	is_nullable
order_id	integer	NO
instrument_id	integer	NO
accepted_on	date	NO
due_on	date	NO
status	text	NO
complaint	text	NO
closed_on	date	YES

(7 rows)

order_id	status	accepted_on	closed_on
1	in_progress	2026-09-01	

2		accepted		2026-09-05		
3		closed		2026-08-10		2026-08-29
4		in_progress		2026-09-02		
5		ready		2026-09-08		
6		cancelled		2026-08-20		
7		accepted		2026-09-10		
8		closed		2026-08-01		2026-08-10

(8 rows)

Листинг 9 — Структура и данные таблицы «restoration_order» после модификации

6 Заполнение базы данных

Код внесения данных приведён в листинге 10. Каждая из восьми самостоятельных таблиц содержит по восемь кортежей, ассоциативная таблица «order_work» — одиннадцать. Данные подобраны так, чтобы проверить все сценарии представлений: у заказа № 1 две работы и две оплаты, у заказа № 2 нет оплат, заказ № 6 отменён, заказы № 3 и № 8 закрыты, а инструмент INS-001 имеет два заказа — закрытый и активный.

Идентификаторы заданы явно, чтобы ссылки в скрипте были читаемыми. Явные значения не продвигают identity-последовательности, поэтому в конце скрипта они устанавливаются на максимальные значения ключей. Без этого следующая вставка без указания ключа получила бы идентификатор 1 и нарушила бы первичный ключ.

```
-- Вымышленные демонстрационные данные:
-- по 8 кортежей в каждой самостоятельной таблице.
BEGIN;

INSERT INTO customer (customer_id, full_name, phone, email)
VALUES
  (1, 'Кузнецова Анна Сергеевна', '+7 900 000-00-01',
   'anna@example.test'),
  (2, 'Орлов Михаил Андреевич', '+7 900 000-00-02',
   'mikhail@example.test'),
  (3, 'Лебедева Ольга Викторовна', '+7 900 000-00-03', NULL),
  (4, 'Соколов Дмитрий Павлович', '+7 900 000-00-04',
   'sokolov@example.test'),
  (5, 'Морозова Ирина Алексеевна', '+7 900 000-00-05',
   'irina@example.test'),
  (6, 'Новиков Артём Ильич', '+7 900 000-00-06', NULL),
  (7, 'Фёдорова Мария Константиновна', '+7 900 000-00-07',
   'fedorova@example.test'),
  (8, 'Егоров Сергей Николаевич', '+7 900 000-00-08',
   'egorov@example.test');

INSERT INTO instrument_type (type_id, name, description)
VALUES
  (1, 'Скрипка', 'Смычковый струнный инструмент'),
  (2, 'Классическая гитара', 'Щипковый, нейлоновые струны'),
  (3, 'Альт', 'Смычковый инструмент, крупнее скрипки'),
```

```

(4, 'Виолончель', 'Басово-теноровый смычковый инструмент'),
(5, 'Контрабас', 'Самый крупный смычковый инструмент'),
(6, 'Мандолина', 'Щипковый инструмент с парными струнами'),
(7, 'Балалайка', 'Русский народный щипковый инструмент'),
(8, 'Домра', 'Народный щипковый с округлым корпусом');

INSERT INTO instrument (instrument_id, customer_id, type_id,
                        inventory_no, name, maker, serial_no)
VALUES
(1, 1, 1, 'INS-001', 'Скрипка ученическая 4/4',
  'Cremona', NULL),
(2, 2, 2, 'INS-002', 'Гитара классическая',
  'Yamaha', 'DEMO-C40'),
(3, 3, 3, 'INS-003', 'Альт концертный 16"',
  'Hofner', 'DEMO-H16'),
(4, 4, 4, 'INS-004', 'Виолончель 4/4',
  'Strunal', NULL),
(5, 5, 5, 'INS-005', 'Контрабас оркестровый 3/4',
  'Cremona', 'DEMO-SB3'),
(6, 6, 6, 'INS-006', 'Мандолина неаполитанская',
  NULL, NULL),
(7, 7, 7, 'INS-007', 'Балалайка прима',
  'Мастер Е. Зуев', NULL),
(8, 8, 8, 'INS-008', 'Домра малая',
  NULL, 'DEMO-D01');

```

```

INSERT INTO employee (employee_id, full_name,
                      specialization, work_email)
VALUES
(1, 'Смирнов Павел Олегович', 'Смычковые инструменты',
  'p.smirnov@example.test'),
(2, 'Волкова Елена Игоревна', 'Щипковые инструменты',
  'e.volkova@example.test'),
(3, 'Козлов Андрей Викторович', 'Крупные смычковые',
  'a.kozlov@example.test'),
(4, 'Попова Наталья Юрьевна', 'Лакокрасочные покрытия',
  'n.porova@example.test'),
(5, 'Васильев Игорь Сергеевич', 'Народные инструменты',
  'i.vasiliev@example.test'),
(6, 'Зайцева Дарья Андреевна', 'Струны и фурнитура',
  'd.zaitseva@example.test'),
(7, 'Павлов Роман Евгеньевич', 'Гриффы и подставки',
  'r.pavlov@example.test'),

```

```

(8, 'Семенова Алина Петровна', 'Диагностика и оценка',
  'a.semenova@example.test');

INSERT INTO service (service_id, name, description,
                    base_price_kopecks)
VALUES
(1, 'Склейка трещины деки',
  'Восстановление целостности деки', 500000),
(2, 'Замена подставки',
  'Подгонка и установка подставки', 200000),
(3, 'Регулировка грифа',
  'Устранение дребезжания струн', 300000),
(4, 'Перетяжка струн',
  'Замена комплекта струн и настройка', 150000),
(5, 'Реставрация лака',
  'Восстановление лакового покрытия', 800000),
(6, 'Замена колков',
  'Подбор и установка колковой механики', 250000),
(7, 'Переклейка обечайки',
  'Устранение отслоения обечайки', 600000),
(8, 'Диагностика',
  'Осмотр и дефектная ведомость', 100000);

INSERT INTO restoration_order (order_id, instrument_id,
                              accepted_on, due_on, status, complaint, closed_on)
VALUES
(1, 1, '2026-09-01', '2026-09-20', 'in_progress',
  'Трещина верхней деки, износ подставки', NULL),
(2, 2, '2026-09-05', '2026-09-25', 'accepted',
  'Дребезжание струн', NULL),
(3, 3, '2026-08-10', '2026-08-30', 'closed',
  'Потёртости лака на нижней деке', '2026-08-29'),
(4, 4, '2026-09-02', '2026-09-22', 'in_progress',
  'Отслоение обечайки, изношены струны', NULL),
(5, 5, '2026-09-08', '2026-09-18', 'ready',
  'Проворачиваются колки', NULL),
(6, 6, '2026-08-20', '2026-09-05', 'cancelled',
  'Нестрой инструмента', NULL),
(7, 7, '2026-09-10', '2026-10-01', 'accepted',
  'Трещина деки у подгрифника', NULL),
(8, 1, '2026-08-01', '2026-08-10', 'closed',
  'Замена изношенной подставки', '2026-08-10');

```

```
INSERT INTO order_work (order_id, line_no, service_id,  
                        employee_id, agreed_price_kopecks, status)
```

```
VALUES
```

```
(1, 1, 1, 1, 550000, 'in_progress'),  
(1, 2, 2, 7, 200000, 'planned'),  
(2, 1, 3, 2, 300000, 'planned'),  
(3, 1, 5, 4, 750000, 'done'),  
(4, 1, 7, 3, 600000, 'in_progress'),  
(4, 2, 4, 6, 150000, 'planned'),  
(5, 1, 6, 6, 250000, 'done'),  
(6, 1, 8, 8, 100000, 'cancelled'),  
(7, 1, 8, 8, 100000, 'done'),  
(7, 2, 1, 5, 450000, 'planned'),  
(8, 1, 2, 1, 150000, 'done');
```

```
INSERT INTO inspection (inspection_id, order_id, employee_id,  
                        inspected_on, conclusion)
```

```
VALUES
```

```
(1, 1, 1, '2026-09-01',  
        'Продольная трещина деки длиной 35 мм'),  
(2, 2, 2, '2026-09-05',  
        'Требуется регулировка прогиба грифа'),  
(3, 1, 1, '2026-09-07',  
        'Трещина стабильна, нужна установка подставки'),  
(4, 3, 4, '2026-08-11',  
        'Лак нижней деки стёрт на площади 20 см²'),  
(5, 4, 3, '2026-09-02',  
        'Обечайка отслоилась на участке 12 см'),  
(6, 5, 6, '2026-09-08',  
        'Колковая механика изношена, нужна замена'),  
(7, 7, 8, '2026-09-10',  
        'Трещина деки у подгрифника длиной 20 мм'),  
(8, 8, 1, '2026-08-01',  
        'Подставка деформирована, замена обязательна');
```

```
INSERT INTO payment (payment_id, order_id, paid_on,  
                     amount_kopecks, method, receipt_no)
```

```
VALUES
```

```
(1, 1, '2026-09-01', 200000, 'card', 'DEMO-001'),  
(2, 1, '2026-09-07', 100000, 'transfer', 'DEMO-002'),  
(3, 8, '2026-08-10', 150000, 'cash', 'DEMO-003'),  
(4, 3, '2026-08-10', 300000, 'card', 'DEMO-004'),  
(5, 3, '2026-08-29', 450000, 'card', 'DEMO-005'),
```

```

(6, 4, '2026-09-02', 300000, 'transfer', 'DEMO-006'),
(7, 5, '2026-09-08', 250000, 'cash', 'DEMO-007'),
(8, 7, '2026-09-10', 100000, 'card', 'DEMO-008');

-- Явные идентификаторы не сдвигают последовательности.
DO $$
DECLARE
    t record;
BEGIN
    FOR t IN SELECT * FROM (VALUES
        ('customer', 'customer_id'),
        ('instrument_type', 'type_id'),
        ('instrument', 'instrument_id'),
        ('employee', 'employee_id'),
        ('service', 'service_id'),
        ('restoration_order', 'order_id'),
        ('inspection', 'inspection_id'),
        ('payment', 'payment_id')
    ) AS v(tab, col) LOOP
        EXECUTE format(
            'SELECT setval(pg_get_serial_sequence(%L, %L), '
            '(SELECT max(%I) FROM %I))',
            t.tab, t.col, t.col, t.tab);
    END LOOP;
END $$;

COMMIT;

```

Листинг 10 — Внесение данных («sql/05_data.sql»)

Количество строк в таблицах после загрузки приведено в листинге 11.

table_name	rows
customer	8
instrument_type	8
instrument	8
employee	8
restoration_order	8
service	8
order_work	11
inspection	8
payment	8

(9 rows)

Листинг 11 — Количество кортежей в таблицах

7 Создание представлений

Четыре представления из пункта 5 первой работы перенесены в PostgreSQL инструкциями «CREATE VIEW» (листинг 12) [5]. Для приёмщика созданы реестр заказов «v_reception_orders» и сводка расчётов «v_reception_balances», для реставратора — очередь работ «v_restorer_tasks» и история осмотров «v_restorer_inspections». Представления реставратора не содержат контактов заказчика и денежных сумм.

В «v_reception_balances» стоимость работ и сумма оплат агрегируются отдельно до соединения с заказом. При прямом соединении двух работ с двумя оплатами одного заказа получились бы четыре строки и обе суммы были бы завышены вдвое.

```
-- Представления из п. 5 ЛР1, перенесённые в PostgreSQL.
BEGIN;

-- Приёмщик: карточки заказов и контакты для уведомлений.
CREATE VIEW v_reception_orders AS
SELECT o.order_id, c.full_name AS customer_name, c.phone,
       i.inventory_no, i.name AS instrument_name,
       t.name AS instrument_type,
       o.accepted_on, o.due_on, o.status
FROM restoration_order o
JOIN instrument i      ON i.instrument_id = o.instrument_id
JOIN customer c        ON c.customer_id = i.customer_id
JOIN instrument_type t ON t.type_id = i.type_id;

-- Приёмщик: стоимость, оплаты и остаток. Суммы считаются
-- до соединения, чтобы работы и оплаты не перемножались.
CREATE VIEW v_reception_balances AS
WITH costs AS (
    SELECT order_id, sum(agreed_price_kopecks) AS total
    FROM order_work
    WHERE status <> 'cancelled'
    GROUP BY order_id
), paid AS (
    SELECT order_id, sum(amount_kopecks) AS total
    FROM payment
    GROUP BY order_id
)
SELECT o.order_id, c.full_name AS customer_name,
```

```

        coalesce(costs.total, 0) AS agreed_kopecks,
        coalesce(paid.total, 0) AS paid_kopecks,
        coalesce(costs.total, 0) - coalesce(paid.total, 0)
            AS balance_kopecks
FROM restoration_order o
JOIN instrument i ON i.instrument_id = o.instrument_id
JOIN customer c ON c.customer_id = i.customer_id
LEFT JOIN costs ON costs.order_id = o.order_id
LEFT JOIN paid ON paid.order_id = o.order_id;

-- Реставратор: очередь работ без контактов и финансов.
CREATE VIEW v_restorer_tasks AS
SELECT w.employee_id, e.full_name AS restorer_name,
       w.order_id, w.line_no, i.inventory_no,
       i.name AS instrument_name, s.name AS service_name,
       o.due_on, w.status AS work_status
FROM order_work w
JOIN employee e ON e.employee_id = w.employee_id
JOIN restoration_order o ON o.order_id = w.order_id
JOIN instrument i ON i.instrument_id = o.instrument_id
JOIN service s ON s.service_id = w.service_id
WHERE w.status IN ('planned', 'in_progress')
      AND o.status IN ('accepted', 'in_progress');

-- Реставратор: история осмотров с авторством заключений.
CREATE VIEW v_restorer_inspections AS
SELECT x.inspection_id, x.order_id, i.inventory_no,
       i.name AS instrument_name, x.inspected_on,
       e.employee_id, e.full_name AS inspector_name,
       x.conclusion
FROM inspection x
JOIN employee e ON e.employee_id = x.employee_id
JOIN restoration_order o ON o.order_id = x.order_id
JOIN instrument i ON i.instrument_id = o.instrument_id;

COMMIT;

```

Листинг 12 — Представления («sql/06_views.sql»)

Результаты запросов к представлениям приведены в листинге 13. Для компактности выбраны основные столбцы, от ФИО оставлена фамилия, суммы выведены в копейках. Сводка расчётов совпадает с ручным расчётом: для заказа № 1 стоимость 7 500 руб. (две работы), оплачено 3 000 руб. (две

оплаты), остаток 4 500 руб.; отменённая работа заказа № 6 в стоимость не входит. В очереди реставраторов нет работ закрытого заказа № 3 и выполненных работ.

id	customer	inv	due_on	status
5	Морозова	INS-005	2026-09-18	ready
1	Кузнецова	INS-001	2026-09-20	in_progress
4	Соколов	INS-004	2026-09-22	in_progress
2	Орлов	INS-002	2026-09-25	accepted
7	Фёдорова	INS-007	2026-10-01	accepted

(5 rows)

id	customer	agreed	paid	balance
1	Кузнецова	750000	300000	450000
2	Орлов	300000	0	300000
3	Лебедева	750000	750000	0
4	Соколов	750000	300000	450000
5	Морозова	250000	250000	0
6	Новиков	0	0	0
7	Фёдорова	550000	100000	450000
8	Кузнецова	150000	150000	0

(8 rows)

restorer	id	ln	service_name	status
Смирнов	1	1	Склейка трещины деки	in_progress
Волкова	2	1	Регулировка грифа	planned
Козлов	4	1	Переклейка обечайки	in_progress
Васильев	7	2	Склейка трещины деки	planned
Зайцева	4	2	Перетяжка струн	planned
Павлов	1	2	Замена подставки	planned

(6 rows)

insp	id	inv	inspected_on	inspector
8	8	INS-001	2026-08-01	Смирнов
1	1	INS-001	2026-09-01	Смирнов
3	1	INS-001	2026-09-07	Смирнов
2	2	INS-002	2026-09-05	Волкова
4	3	INS-003	2026-08-11	Попова
5	4	INS-004	2026-09-02	Козлов

6		5		INS-005		2026-09-08		Зайцева
7		7		INS-007		2026-09-10		Семенова

(8 rows)

Листинг 13 — Результаты запросов к представлениям

8 Проверка ограничений целостности

Для проверки связей и ограничений выполнены шесть заведомо некорректных операций. Каждая выполняется в отдельном блоке с перехватом исключения; если бы СУБД приняла операцию, сценарий завершился бы ошибкой. Результат приведён в листинге 14: все операции отклонены с кодами 23505 (нарушение уникальности), 23503 (нарушение внешнего ключа) и 23514 (нарушение проверочного ограничения), причём указано именно ожидаемое ограничение.

probe		rejected_by
-----+-----		
второй активный заказ		23505 order_one_active_uq
удаление заказчика		23503 instrument_customer_fk
несуществующий тип		23503 instrument_type_fk
закрытие без даты		23514 order_closed_on_check
срок раньше приёма		23514 restoration_order_check
отрицательная оплата		23514 payment_amount_kopecks_check
(6 rows)		

Листинг 14 — Отклонённые некорректные операции

Все результаты воспроизводятся командой «make check» из каталога работы: сценарий пересоздаёт базу данных, выполняет шесть SQL-скриптов по порядку и сохраняет выводы в каталог «verification».

ЗАКЛЮЧЕНИЕ

В СУБД PostgreSQL 14 реализована база данных мастерской реставрации музыкальных инструментов по схеме из первой работы. Созданы девять таблиц, девять внешних ключей с запретом удаления используемых записей, индексы для атрибутов соединений и фильтрации, выполнена модификация структуры с добавлением столбца и ограничения. Таблицы заполнены демонстрационными данными, созданы четыре представления для приёмщика и реставратора.

Проверки подтвердили наличие связей и индексов, использование индексов планировщиком, корректность расчётов в представлениях и отклонение некорректных операций. Полученная база данных используется как основа для разграничения доступа, журналирования и шифрования в лабораторной работе № 3.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические указания по лабораторной работе № 2 «Реализация БД в рамках СУБД» по дисциплине «Информационная безопасность баз данных».
2. PostgreSQL 14 Documentation. Chapter 5. Data Definition [Электронный ресурс]. URL: <https://www.postgresql.org/docs/14/ddl.html> (дата обращения: 19.09.2026).
3. PostgreSQL 14 Documentation. Chapter 11. Indexes [Электронный ресурс]. URL: <https://www.postgresql.org/docs/14/indexes.html> (дата обращения: 19.09.2026).
4. PostgreSQL 14 Documentation. ALTER TABLE [Электронный ресурс]. URL: <https://www.postgresql.org/docs/14/sql-altertable.html> (дата обращения: 19.09.2026).
5. PostgreSQL 14 Documentation. CREATE VIEW [Электронный ресурс]. URL: <https://www.postgresql.org/docs/14/sql-createview.html> (дата обращения: 19.09.2026).